



AWS CloudFormation Lab: Create a VPC



Objective

Use AWS CloudFormation to **automate the creation of a VPC** with:

- 1 VPC
- 2 Public Subnets
- 2 Private Subnets
- 1 Internet Gateway
- Route Tables and Routes



Prerequisites

- AWS Account
- IAM user with **CloudFormation**, **VPC**, and **EC2** permissions
- AWS CLI or Console access



Step 1: CloudFormation Template

Save the following as **vpc-setup.yaml**:

AWSTemplateFormatVersion: '2010-09-09'

Description: CloudFormation template to create a VPC with public and private subnets

Parameters:

VpcCIDR:

Type: String

Default: 10.0.0.0/16

PublicSubnet1CIDR:

Type: String

Default: 10.0.1.0/24

PublicSubnet2CIDR:

Type: String

Default: 10.0.2.0/24

PrivateSubnet1CIDR:

Type: String

Default: 10.0.3.0/24

PrivateSubnet2CIDR:

Type: String

Default: 10.0.4.0/24

Resources:

MyVPC:

Type: AWS::EC2::VPC

Properties:

CidrBlock: !Ref VpcCIDR

EnableDnsSupport: true

EnableDnsHostnames: true

Tags:

- Key: Name

Value: MyCFN-VPC

InternetGateway:

Type: AWS::EC2::InternetGateway

Properties:

Tags:

- Key: Name

Value: MyCFN-IGW

AttachGateway:

Type: AWS::EC2::VPCGatewayAttachment

Properties:

VpcId: !Ref MyVPC

InternetGatewayId: !Ref InternetGateway

PublicSubnet1:

Type: AWS::EC2::Subnet

Properties:

VpcId: !Ref MyVPC
CidrBlock: !Ref PublicSubnet1CIDR
AvailabilityZone: !Select [0, !GetAZs ""]
MapPublicIpOnLaunch: true
Tags:
- Key: Name
Value: PublicSubnet1

PublicSubnet2:
Type: AWS::EC2::Subnet
Properties:
VpcId: !Ref MyVPC
CidrBlock: !Ref PublicSubnet2CIDR
AvailabilityZone: !Select [1, !GetAZs ""]
MapPublicIpOnLaunch: true
Tags:
- Key: Name
Value: PublicSubnet2

PrivateSubnet1:
Type: AWS::EC2::Subnet
Properties:
VpcId: !Ref MyVPC
CidrBlock: !Ref PrivateSubnet1CIDR
AvailabilityZone: !Select [0, !GetAZs ""]
Tags:
- Key: Name
Value: PrivateSubnet1

PrivateSubnet2:
Type: AWS::EC2::Subnet
Properties:
VpcId: !Ref MyVPC
CidrBlock: !Ref PrivateSubnet2CIDR
AvailabilityZone: !Select [1, !GetAZs ""]
Tags:
- Key: Name
Value: PrivateSubnet2

PublicRouteTable:
Type: AWS::EC2::RouteTable
Properties:
VpcId: !Ref MyVPC
Tags:

- Key: Name
Value: PublicRouteTable

DefaultPublicRoute:

Type: AWS::EC2::Route

DependsOn: AttachGateway

Properties:

RouteTableId: !Ref PublicRouteTable

DestinationCidrBlock: 0.0.0.0/0

GatewayId: !Ref InternetGateway

PublicSubnet1RouteTableAssoc:

Type: AWS::EC2::SubnetRouteTableAssociation

Properties:

SubnetId: !Ref PublicSubnet1

RouteTableId: !Ref PublicRouteTable

PublicSubnet2RouteTableAssoc:

Type: AWS::EC2::SubnetRouteTableAssociation

Properties:

SubnetId: !Ref PublicSubnet2

RouteTableId: !Ref PublicRouteTable

Outputs:

VPCId:

Description: VPC ID

Value: !Ref MyVPC

PublicSubnet1Id:

Value: !Ref PublicSubnet1

PublicSubnet2Id:

Value: !Ref PublicSubnet2

PrivateSubnet1Id:

Value: !Ref PrivateSubnet1

PrivateSubnet2Id:

Value: !Ref PrivateSubnet2

Step 2: Launch the Stack

Option 1: Using AWS Console

1. Go to [CloudFormation Console](#)
 2. Click **Create stack** → **With new resources (standard)**
 3. Upload the `vpc-setup.yaml` file
 4. Click **Next**, name the stack (e.g., `MyVPCStack`)
 5. Accept default parameters or customize
 6. Click **Next** → **Create stack**
-

Option 2: Using AWS CLI

```
aws cloudformation create-stack \  
--stack-name MyVPCStack \  
--template-body file://vpc-setup.yaml \  
--capabilities CAPABILITY_NAMED_IAM
```

Step 3: Verify

- Go to **VPC Console** → Check:
 - VPC
 - Subnets
 - Route Tables
 - Internet Gateway
 - All resources should be tagged with `MyCFN-*`
-

Step 4: Cleanup

To delete all created resources:

Console:

- CloudFormation → Select the stack → **Delete**

CLI:

```
aws cloudformation delete-stack --stack-name MyVPCStack
```

Summary

Resource	Created
VPC	✓
Subnets (Public/Private)	✓
IGW + Route Table	✓
Associations	✓
Fully Automated	✓
